

Research Notes: Beyond Softmax

New Activation Functions for Classification and Regression

Your Name

April 20, 2026

Reading Material for Personal Use

Contents

1	Introduction	2
2	What Softmax Is and Why It Dominates	2
2.1	Why Softmax Succeeded	2
2.2	Limitations of Softmax	2
3	Historical Precedent: Sigmoid to ReLU	3
3.1	Sigmoid	3
3.2	ReLU	3
3.3	The Lesson	3
3.4	The Question	3
4	The Power Softmax Family	3
4.1	Definition	3
4.2	Why This Makes Sense	4
4.3	Mathematical Properties to Verify	4
4.4	Gradient Derivation	4
4.5	Open Questions About Power Softmax	4
5	Beyond Classification: Regression	5
5.1	Why Softmax Doesn't Work for Regression	5
5.2	Existing Bridge: Mixture Density Networks	5
5.3	Idea: Softmax Over Bins	5
5.4	Can Power Softmax Do Better Here?	5
5.5	Alternative: Direct Regression Without Binning	5
6	Other Function Families to Consider	6
6.1	The Exponential-Power Hybrid	6
6.2	The Rational Softmax	6
6.3	The Entmax Family (Existing Work)	6
6.4	The Trigonometric Softmax	6

7	Theoretical Questions a Mathematician Could Answer	7
7.1	Generalization Bounds	7
7.2	Convergence Rates	7
7.3	Information-Theoretic Interpretation	7
7.4	The Limit $\gamma \rightarrow 0$ and $\gamma \rightarrow \infty$	7
8	Experimental Plan (To Be Refined)	7
8.1	Classification Experiments	7
8.2	Regression Experiments	8
9	Next Steps for Me	8
10	Open Questions for My Supervisor	8
11	Preliminary References	8

1 Introduction

This document summarizes my current thinking about how to approach the problem my supervisor gave me: invent a completely new activation function that can outperform softmax in some domain, possibly regression.

The key points from our conversation:

- Do not modify delta or existing AS-Softmax parameters
- Think from first principles as a mathematician
- Look at what sigmoid $\rightarrow$ ReLU taught us
- Consider regression as an alternative domain to classification

2 What Softmax Is and Why It Dominates

The softmax function is defined as:

$$p_i = \frac{e^{o_i}}{\sum_{j=1}^n e^{o_j}}, \quad i = 1, \dots, n$$

where $o \in \mathbb{R}^n$ are logits (the raw outputs of a neural network) and p is a probability distribution over n classes.

2.1 Why Softmax Succeeded

1. **Differentiability:** The gradient is simple and elegant:

$$\frac{\partial p_i}{\partial o_j} = p_i(\delta_{ij} - p_j)$$

2. **Natural probability interpretation:** Outputs are in $(0, 1)$ and sum to 1.
3. **Works with cross-entropy loss:** For a target class t ,

$$\mathcal{L} = -\log p_t$$

The gradient becomes p_j for $j \neq t$ and $p_t - 1$ for $j = t$.

4. **Convexity in logits:** The loss is convex in o , which helps optimization.

2.2 Limitations of Softmax

Despite its dominance, softmax has well-known problems:

1. **Extreme confidence:** For even moderately large logits, $p_t \approx 1$ very quickly. This leads to overconfidence and overfitting.
2. **No sparsity:** Every class gets a positive probability, even those with very negative logits.
3. **Exponential cost:** Computing e^{o_i} is expensive and can overflow.
4. **Train-test mismatch:** Training pushes $p_t \rightarrow 1$ but test only requires $p_t > p_j$ for all $j \neq t$ (as shown in AS-Softmax).

3 Historical Precedent: Sigmoid to ReLU

Your supervisor mentioned this. Let me make the analogy explicit.

3.1 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Problems: Vanishing gradients for large $|x|$, outputs not zero-centered.

3.2 ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Advantages: No vanishing gradient for positive x , computationally trivial, induces sparsity.

3.3 The Lesson

A simpler, piecewise-linear function replaced a smooth exponential function for hidden layers because it solved a specific problem (vanishing gradients).

3.4 The Question

Can we find a similarly simpler function to replace softmax for the final layer? What problem would it solve? Possible candidates:

- Sparsity (zero probabilities for many classes)
- Better confidence calibration
- Faster computation
- Working naturally with regression targets

4 The Power Softmax Family

Here is my first concrete proposal to investigate.

4.1 Definition

$$p_i = \frac{\max(0, o_i)^\gamma}{\sum_{j=1}^n \max(0, o_j)^\gamma}, \quad \gamma \geq 0$$

where $x_+ = \max(0, x)$ denotes the positive part.

4.2 Why This Makes Sense

1. **Computational simplicity:** Powers are cheaper than exponentials.
2. **Natural sparsity:** If $o_i \leq 0$, then $p_i = 0$ exactly.
3. **Tunable behavior:** The parameter γ controls how sharply the function responds to large logits.
4. **Familiar special cases:**
 - $\gamma = 0$: Uniform distribution (all positive logits treated equally)
 - $\gamma = 1$: Normalized positive part
 - $\gamma = 2$: Squared normalized positive part
 - $\gamma \rightarrow \infty$: Argmax (one-hot)

4.3 Mathematical Properties to Verify

- Non-negativity: Yes, by construction.
- Sum to 1: Yes, by normalization.
- Monotonicity: If $o_i > o_j$, then $p_i > p_j$ (for $\gamma > 0$).
- Differentiability: Almost everywhere. At $o_i = 0$, the function is not differentiable (like ReLU), but this is usually fine.

4.4 Gradient Derivation

Let $S = \sum_j (o_j^+)^{\gamma}$. Then $p_i = (o_i^+)^{\gamma} / S$.

For $o_i > 0$:

$$\frac{\partial p_i}{\partial o_i} = \frac{\gamma o_i^{\gamma-1}}{S} - p_i \cdot \frac{\gamma o_i^{\gamma-1}}{S}$$

For $o_i \leq 0$, $p_i = 0$ and the gradient is 0.

For cross-entropy loss $\mathcal{L} = -\log p_t$, we get:

$$\frac{\partial \mathcal{L}}{\partial o_i} = \begin{cases} \frac{\gamma o_i^{\gamma-1}}{S} \cdot \frac{p_t-1}{p_t}, & i = t, o_i > 0 \\ \frac{\gamma o_i^{\gamma-1}}{S}, & i \neq t, o_i > 0 \\ 0, & o_i \leq 0 \end{cases}$$

This is more complex than softmax's gradient but still tractable.

4.5 Open Questions About Power Softmax

1. What is the optimal γ ? Does it depend on the dataset or task?
2. Does the sparsity actually help? (Fewer classes to compute, less overfitting)
3. Is the gradient stable for $\gamma < 1$? (Division by small numbers)
4. How does calibration compare to softmax?

5 Beyond Classification: Regression

Your supervisor specifically mentioned regression as an alternative domain. Softmax is rarely used for regression. This could be an opportunity.

5.1 Why Softmax Doesn't Work for Regression

Softmax produces a probability distribution over discrete classes. Regression requires a continuous output. You cannot directly use softmax to predict, say, the price of a house.

5.2 Existing Bridge: Mixture Density Networks

Mixture density networks use softmax to output mixing coefficients for a mixture of Gaussians. For example:

$$p(y|x) = \sum_{k=1}^K \pi_k(x) \cdot \mathcal{N}(y|\mu_k(x), \sigma_k^2(x))$$

where $\pi_k(x) = \text{softmax}(o_k)$.

This works, but softmax is just one piece.

5.3 Idea: Softmax Over Bins

Discretize the target space into n bins. Use softmax to predict a probability over bins. Then take a weighted average:

$$\hat{y} = \sum_{i=1}^n v_i p_i$$

where v_i are bin centers (e.g., equally spaced between 0 and 1).

This is essentially a soft version of histogram regression.

5.4 Can Power Softmax Do Better Here?

For regression, we might want:

- Sharper distributions (to concentrate mass near the true value)
- Sparsity (only a few bins should have non-zero probability)
- Better calibration (the predicted distribution should reflect true uncertainty)

Power Softmax with $\gamma > 1$ gives sharper, sparser distributions than standard softmax. This could be beneficial.

5.5 Alternative: Direct Regression Without Binning

What if we design a function that maps logits directly to a continuous value without binning? For example:

$$\hat{y} = \frac{\sum_i o_i^+ \cdot w_i}{\sum_j o_j^+}$$

where w_i are learnable weights. This is like a softmax-weighted average but using positive part instead of exponential.

This is speculative but worth exploring.

6 Other Function Families to Consider

Power softmax is just one idea. Here are others worth exploring.

6.1 The Exponential-Power Hybrid

$$p_i = \frac{e^{o_i^p}}{\sum_j e^{o_j^p}}, \quad p \geq 1$$

When $p = 1$, this is standard softmax. When $p > 1$, it exaggerates differences even more. When $0 < p < 1$, it compresses differences. This could control confidence.

6.2 The Rational Softmax

$$p_i = \frac{o_i^+}{\sum_j o_j^+} \quad (\text{already covered as } \gamma = 1)$$

But we could also try:

$$p_i = \frac{o_i^+ + \epsilon}{\sum_j (o_j^+ + \epsilon)}$$

where ϵ is a small constant to avoid division by zero and control sparsity.

6.3 The Entmax Family (Existing Work)

Entmax generalizes softmax using Tsallis entropy:

$$\text{Entmax}_\alpha(o) = \arg \max_{p \in \Delta} p^T o + H_\alpha(p)$$

where H_α is Tsallis entropy. Special cases:

- $\alpha = 1$: Softmax
- $\alpha = 2$: Sparsemax (produces sparse outputs)

We should not ignore this existing work. But perhaps we can find something simpler.

6.4 The Trigonometric Softmax

$$p_i = \frac{\cos(o_i) + 1}{\sum_j (\cos(o_j) + 1)}$$

This is periodic, which could be useful for cyclic targets (angles, time of day). But monotonicity is lost within a period.

7 Theoretical Questions a Mathematician Could Answer

Here are deeper mathematical questions you could explore.

7.1 Generalization Bounds

Can we prove that Power Softmax has better generalization than standard softmax? For softmax, the margin between classes is related to $\log(n - 1)$. For Power Softmax, what is the analogous condition?

7.2 Convergence Rates

Does gradient descent converge faster with Power Softmax? The gradient magnitude is different:

$$\|\nabla \mathcal{L}_{\text{pow}}\| \leq \|\nabla \mathcal{L}_{\text{exp}}\|$$

in some regimes. Could this imply faster convergence?

7.3 Information-Theoretic Interpretation

Softmax maximizes entropy subject to linear constraints. What does Power Softmax optimize? It may correspond to maximizing Tsallis entropy:

$$H_{\alpha}(p) = \frac{1}{\alpha - 1} \left(1 - \sum_i p_i^{\alpha} \right)$$

For $\alpha = 2$, this gives Sparsemax. For Power Softmax with γ , what is the corresponding α ? Deriving this connection would be a significant theoretical contribution.

7.4 The Limit $\gamma \rightarrow 0$ and $\gamma \rightarrow \infty$

As $\gamma \rightarrow 0$, Power Softmax approaches uniform distribution over classes with positive logits. As $\gamma \rightarrow \infty$, it approaches argmax. The transition between these extremes is smooth. Can we characterize the critical γ where the behavior changes qualitatively?

8 Experimental Plan (To Be Refined)

Once we settle on a function to test, here is a rough experimental plan.

8.1 Classification Experiments

- **Datasets:** SST5 (sentiment, 5 classes), Clinc_os (intent, 151 classes), Conll2003 (NER, 9 classes)
- **Baselines:** Softmax, AS-Softmax, Sparsemax
- **Our method:** Power Softmax with various γ
- **Metrics:** Accuracy, F1, calibration error, training time

8.2 Regression Experiments

- **Task:** Bounded regression (e.g., predicting a value in $[0, 1]$)
- **Method:** Discretize into bins, use Power Softmax over bins, weighted average prediction
- **Baselines:** Linear regression, neural network with linear output
- **Metrics:** MSE, MAE, calibration of uncertainty

9 Next Steps for Me

1. Implement Power Softmax in PyTorch (simple, just a few lines)
2. Test on a tiny dataset to see if it even works
3. Compare $\gamma = 1$ (simplest) against softmax
4. If promising, run a γ sweep on one dataset
5. Report findings to supervisor

10 Open Questions for My Supervisor

When I meet with him next, I should ask:

1. Does the Power Softmax direction interest you, or should I explore something else?
2. Should I focus on classification (to directly compete with softmax) or regression (where softmax is rare)?
3. How much mathematical theory do you want? (Generalization bounds? Convergence proofs? Or just empirical results?)
4. What datasets should I prioritize?
5. Is there a specific domain (text, image, audio) you care most about?

11 Preliminary References

1. Lv, Q. et al. (2023). Adaptive Sparse Softmax. IEEE TASLP.
2. Martins, A. & Astudillo, R. (2016). From Softmax to Sparsemax. ICML.
3. Peters, B. et al. (2019). Sparse Sequence-to-Sequence Models. ACL.
4. Szegedy, C. et al. (2016). Rethinking the Inception Architecture. CVPR.